

# CZ2006/CE2006 Software Engineering

## Lecture 21: Software Maintenance

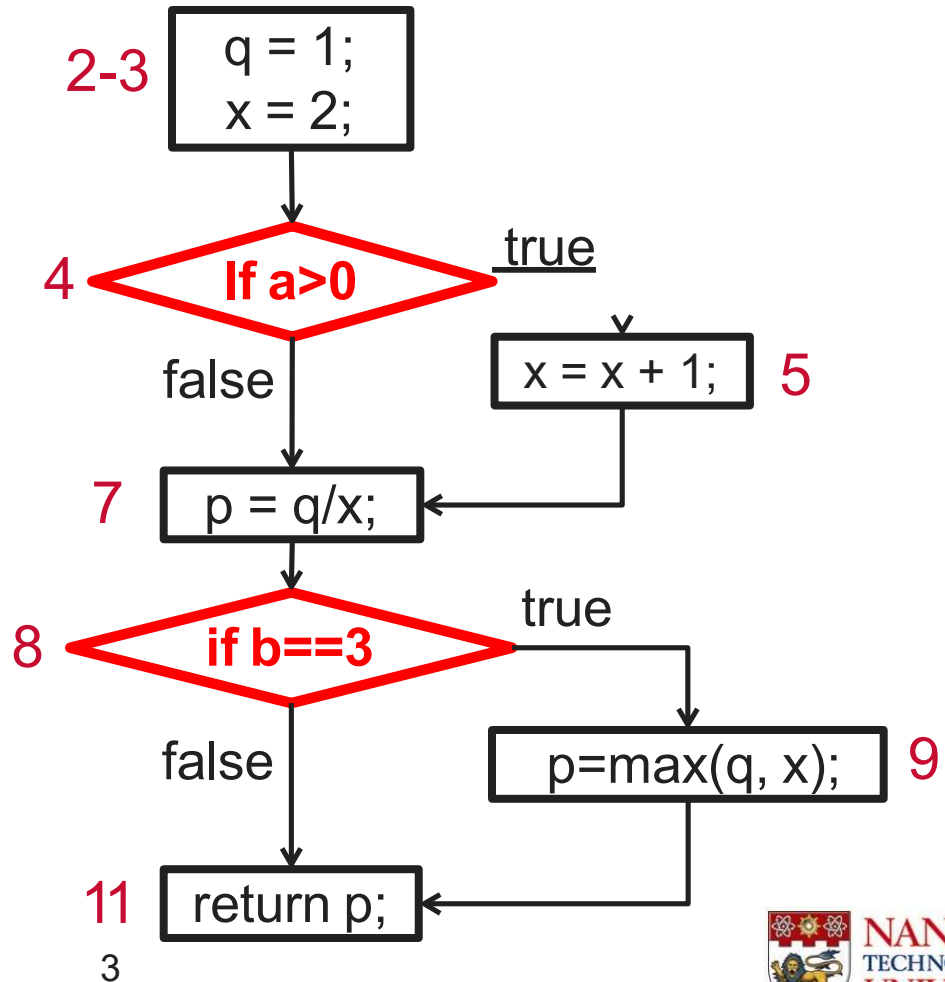
Liu Yang

# **White Box Testing Summary: Control Flow Testing**

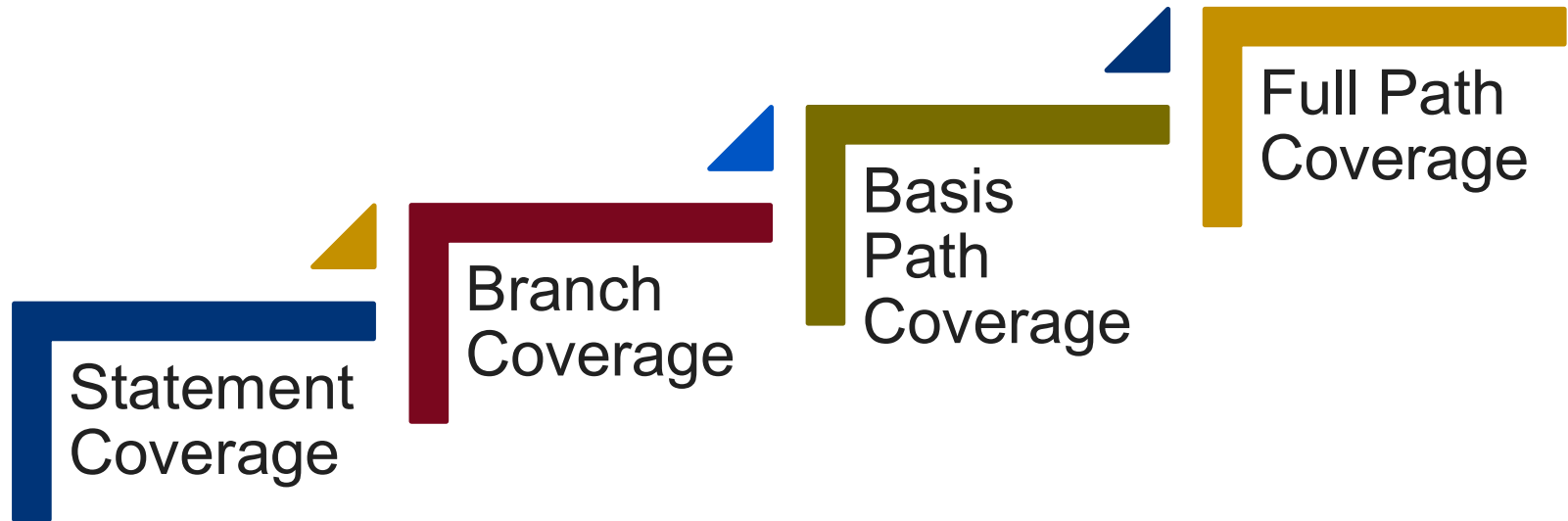
# Control Flow Graph - Example

```
int computeP(int a, int b) {
```

```
1   int q, x, p;  
2   q = 1;  
3   x = 2;  
4   if(a > 0) {  
5       x = x + 1;  
6   }  
7   p = q/x;  
8   if(b == 3) {  
9       p = max(q, x);  
10  }  
11  return p;  
12 }
```



# Coverage Criteria



# White Box Testing: Basis Path Testing Process

- ▶ Construct **control flow graph (CFG)** from the program (code fragment) of the SUT.
- ▶ Identifies **execution paths** through the CFG.
  - ▶ Computes the CFG's **Cyclomatic Complexity (CC)**
  - ▶ Selects a set of **#CC basis paths**
- ▶ Choose **input values** to execute the paths.
- ▶ The **CC** value tells us the upper bound on the size of the basis set, i.e. gives us the minimum number of linearly independent paths (basis paths) we need to find. E.g. if your program has CC value 6, then you've to identify six basic paths to test.

***Basis Path testing guarantees 100% statement and branch coverage***

# White Box Testing: Cyclomatic Complexity (CC) of a CFG

- ▶ Use the following two equations to compute CC
  - ▶  $CC = |\text{edges}| - |\text{nodes}| + 2$
  - ▶  $CC = |\text{decisionpoint}| + 1$ , if all decision points are binary, i.e., one true branch + one false branch

***The two equations compute the same value if all decision points are binary***

# White Box Testing: Cyclomatic Complexity (CC) – Example

```
int computeP(int a, int b) {
```

```
1  int q, x, p;
```

```
2  q = 1;
```

```
3  x = 2;
```

```
4  if(a > 0) {
```

```
5    x = x + 1;
```

```
6  }
```

```
7  p = q/x;
```

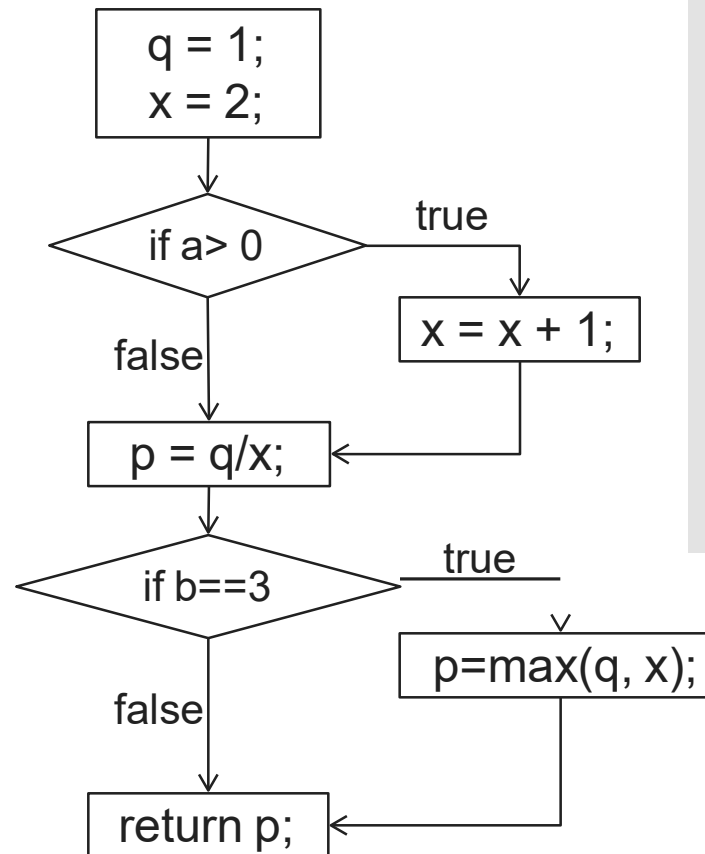
```
8  if(b == 3) {
```

```
9    p = max(q, x);
```

```
10 }
```

```
11 return p;
```

```
}
```



$|edges| = 8$

$|nodes| = 7$

**$CC = 8 - 7 + 2 = 3$**

Or,

$|decisionpoint| = 2$

**$CC = 2 + 1 = 3$**

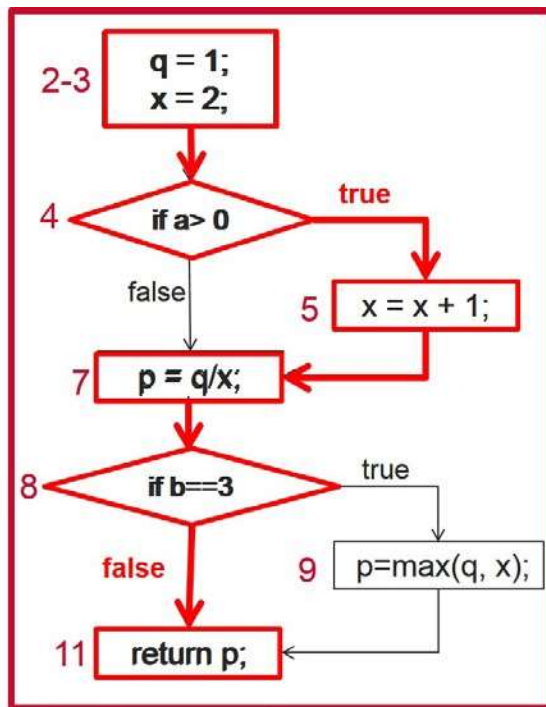
# White Box Testing: Select a Set of Basis Paths

- ▶ Choose a “**baseline**” path
  - ▶ Reasonably “typical” path of execution
  - ▶ Most important path from the tester’s view
- ▶ **Change** the outcome of the **first decision point**, keep the maximum number of other decision points the same
- ▶ **Change** the outcome of the **second decision point**, keep the maximum number of other decision points the same
- ▶ **Change** the outcome of the **third decision point**, and so forth



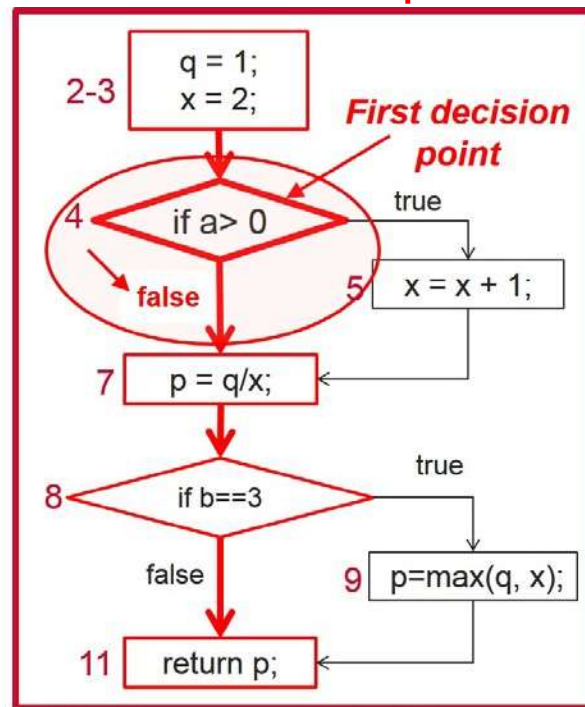
# White Box Testing: Select a Set of Basis Paths

Select baseline path



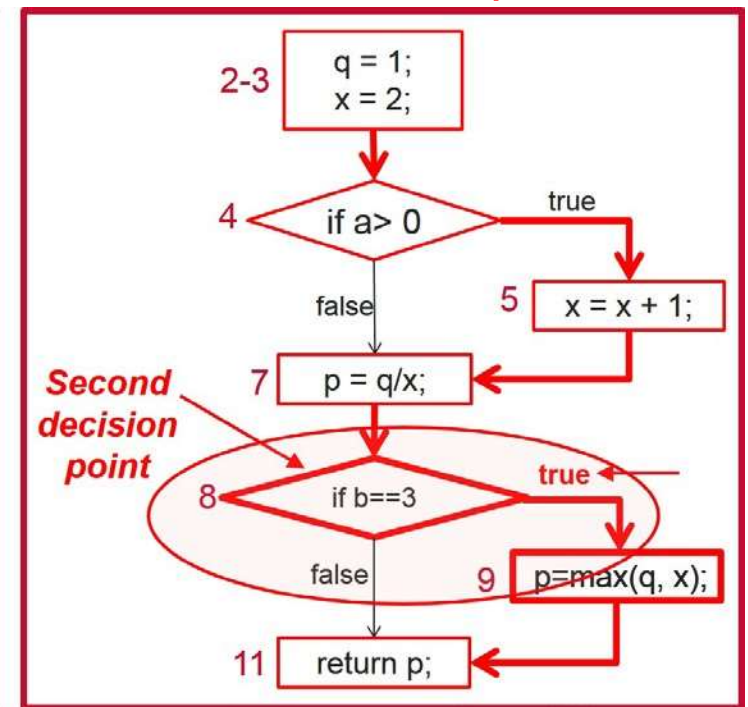
Basis path#1 (baseline):  
**2-3, 4, 5, 7, 8, 11**

Change outcome of first decision point



Basis path#2:  
**2-3, 4, 7, 8, 11**

Change outcome of second decision point



Basis path#3 :  
**2-3, 4, 5, 7, 8, 9, 11**

# White Box Testing: Basis Path Testing

## For independent paths (Basis Paths) –

- The basis set is **not unique**
- There **may be several different basis sets for the given algorithm, based on the baseline path you've chosen**. You may have derived a different basis set
- The basis set **“covers” all the nodes and edges** in the CFG (Basis Path testing guarantees 100% statement and branch coverage)

## For test cases –

- Prepare test cases that will force execution of each path in the basis set, i.e. **choose input parameters (values) to execute each path in the basis set**.
- A basis path **may not be feasible** regardless of input parameters.

# White Box Testing – Dealing with Loop

## 1. To choose basis path (for loop):

- ▶ When **selecting basis paths**, always execute the loop **zero and once (i.e. no need to consider iteration)**.
- ▶ When **selecting baseline paths**, select **false branch first** at loop decision point, i.e., ***do not enter the loop***.
- ▶ Then execute the loop **ONE time** (i.e. the TRUE branch next, for one time).

## 1. For basis path testing (for loop), the test cases may execute the loop **multiple iterations**.

# Testing – Both Testings are Important!

## ▶ Black Box Testing

- ▶ Equivalence class testing (range of values, discrete values)
- ▶ Boundary value testing (only applicable to range of values)

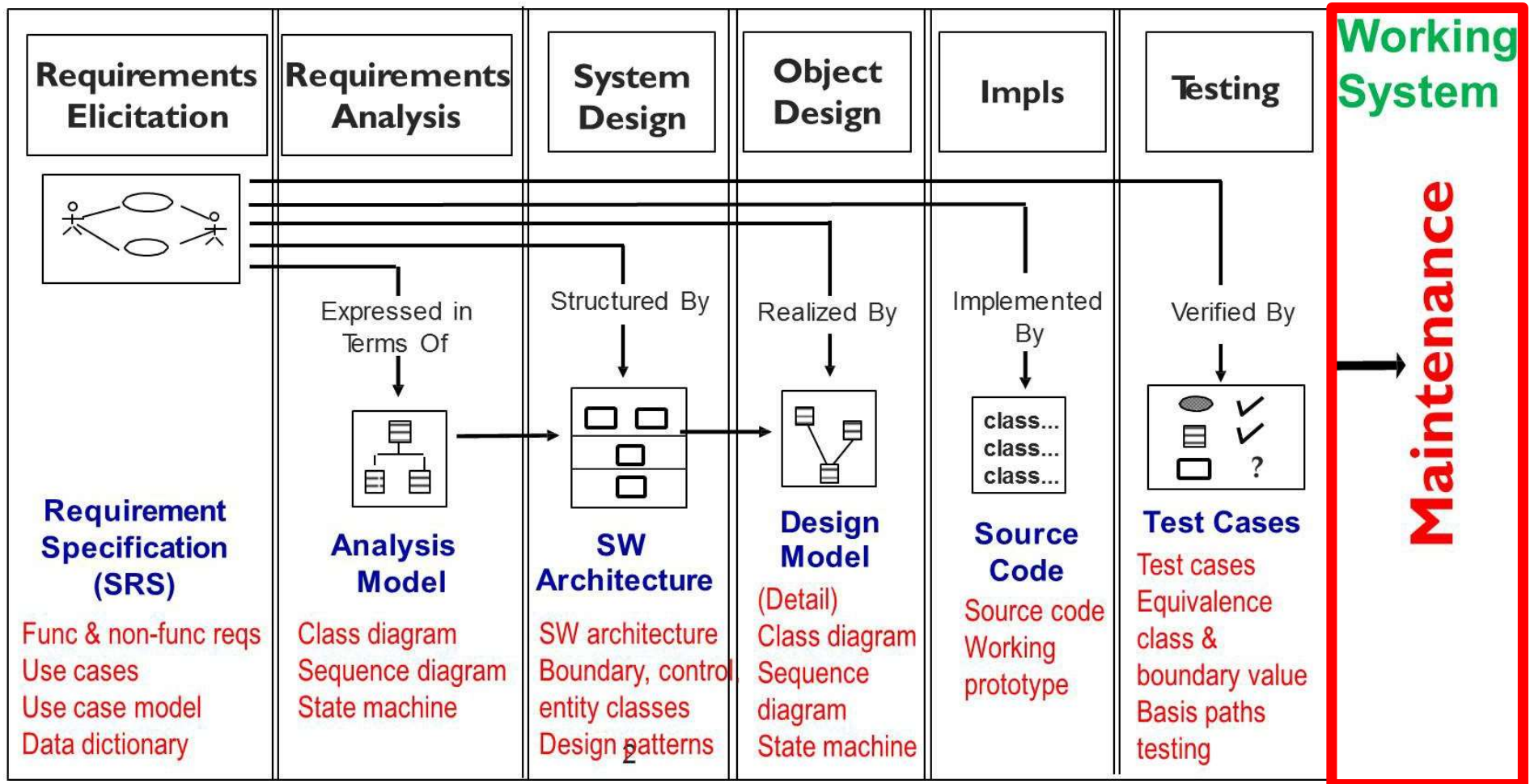
## ▶ White Box Testing

- ▶ Control flow graph (CFG)
- ▶ Statement, branch, basis path, path coverage
- ▶ Basis path testing
  - ▶ Cyclomatic complexity (how many basis paths you need to test)
  - ▶ Select basis paths
  - ▶ Choose input values to execute the paths

*Lecture#16 – Lecture#19*

*Tutorial#9 Q1, Tutorial#10 Q1*

# Software Maintenance



# Causes of Maintenance Problems

- ▶ It is estimated that there are more than 100 billion lines of code in production in the world.\*
- ▶ As much as 80% of it is *unstructured, patched, and badly documented*.\*

\* Hans van Vliet "Software Engineering Principles and Practices" 3rd Edition

# Causes of Maintenance Problems

## ▶ Unstructured code

- ▶ E.g. Numerous GOTO statements – *hard to read, understand, and maintain!* (Spaghetti code)
  - Improve by using structured programming such as IF-ELSE statements.
- ▶ E.g. Poor and inconsistent naming, long procedures, strong coupling, weak cohesion, deeply-nested IF statements, etc.

## ▶ Insufficient domain knowledge

- ▶ To gain a sufficient understanding of a system from its source code.
- ▶ The more spaghetti-like this code is, the less easy it becomes to disentangle it.

## ▶ Insufficient documentation

- ▶ No documentation, out-of-date documentation, or insufficient documentation.



# What is Software Maintenance?

- ▶ **Software maintenance** is defined as (IEEE610, 1990)\*:

*“The process of **modifying** a software system after delivery to **correct faults, improve performance or other attributes, or adapt to a changed environment.**”*

\* IEEE610 (1990). IEEE Standard Glossary of Software Engineering Terminology. IEEE Std 610.12.



# What is Software Maintenance?

- ▶ **Software maintenance is thus concerned with:**
  - ▶ Correcting errors found after the software has been delivered.
  - ▶ Adapting the software to changing requirements, changing environments, etc.
- ▶ Changes are implemented by modifying existing system components and/or adding new components to the system.

# Types of Software Maintenance

- ▶ **Fault repairs:**

- ▶ Changing a system to fix bugs/vulnerabilities and correct deficiencies in the way meets its requirements.

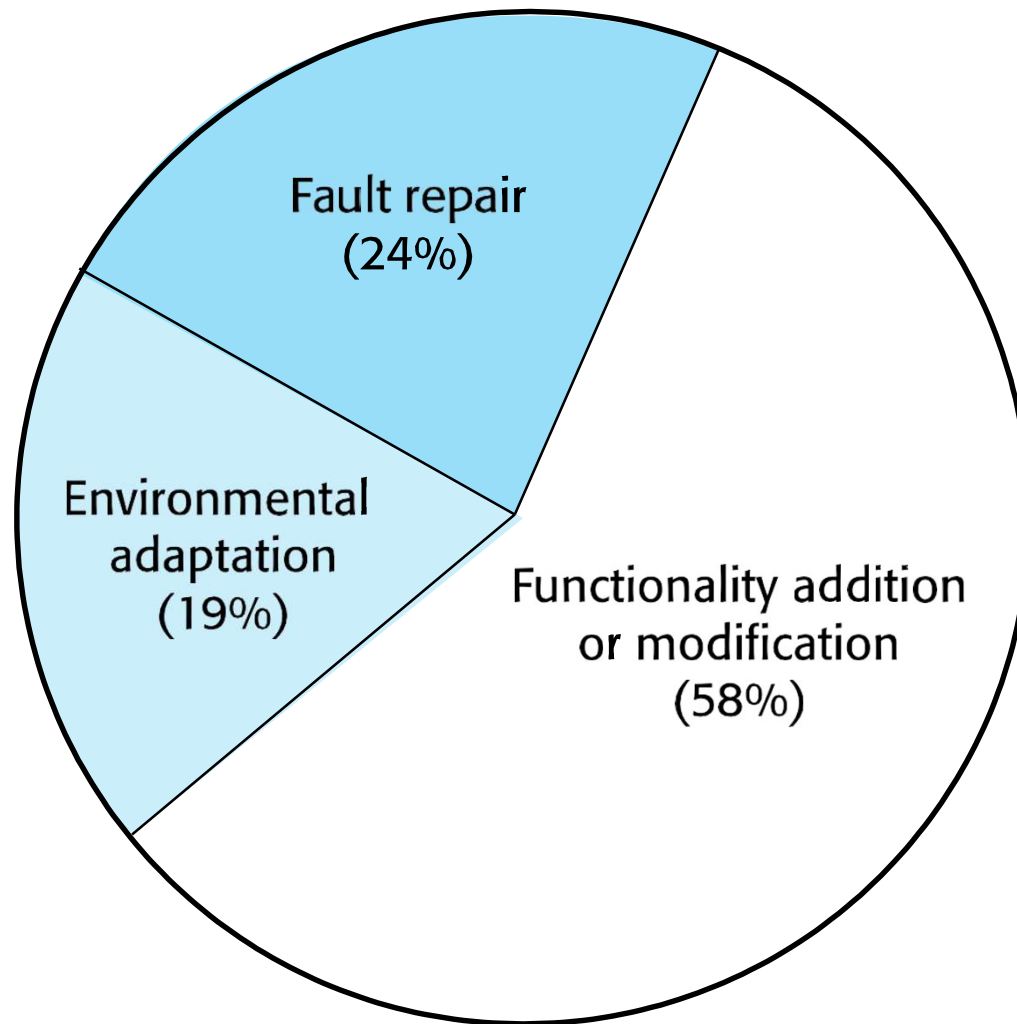
- ▶ **Environmental adaptation:**

- ▶ Maintenance to adapt software to a different operating environment.
- ▶ Changing a system so that it operates in a different environment (computer, OS, etc.) from its initial implementation.

- ▶ **Functionality addition/modification:**

- ▶ Modifying the system to satisfy new requirements.

# Maintenance Effort Distribution



*Adapted from Ian Sommerville "Software Engineering" 9th Edition*

# Software Refactoring

- ▶ **Refactoring** is the process of making improvements to a program to slow down degradation through change. It means modifying (but not adding functionality) a program to:
  - ▶ Improve its structure
  - ▶ Reduce its complexity
  - ▶ Make it easier to understand
- ▶ Refactoring does not change the behavior of the software

# Software Refactoring

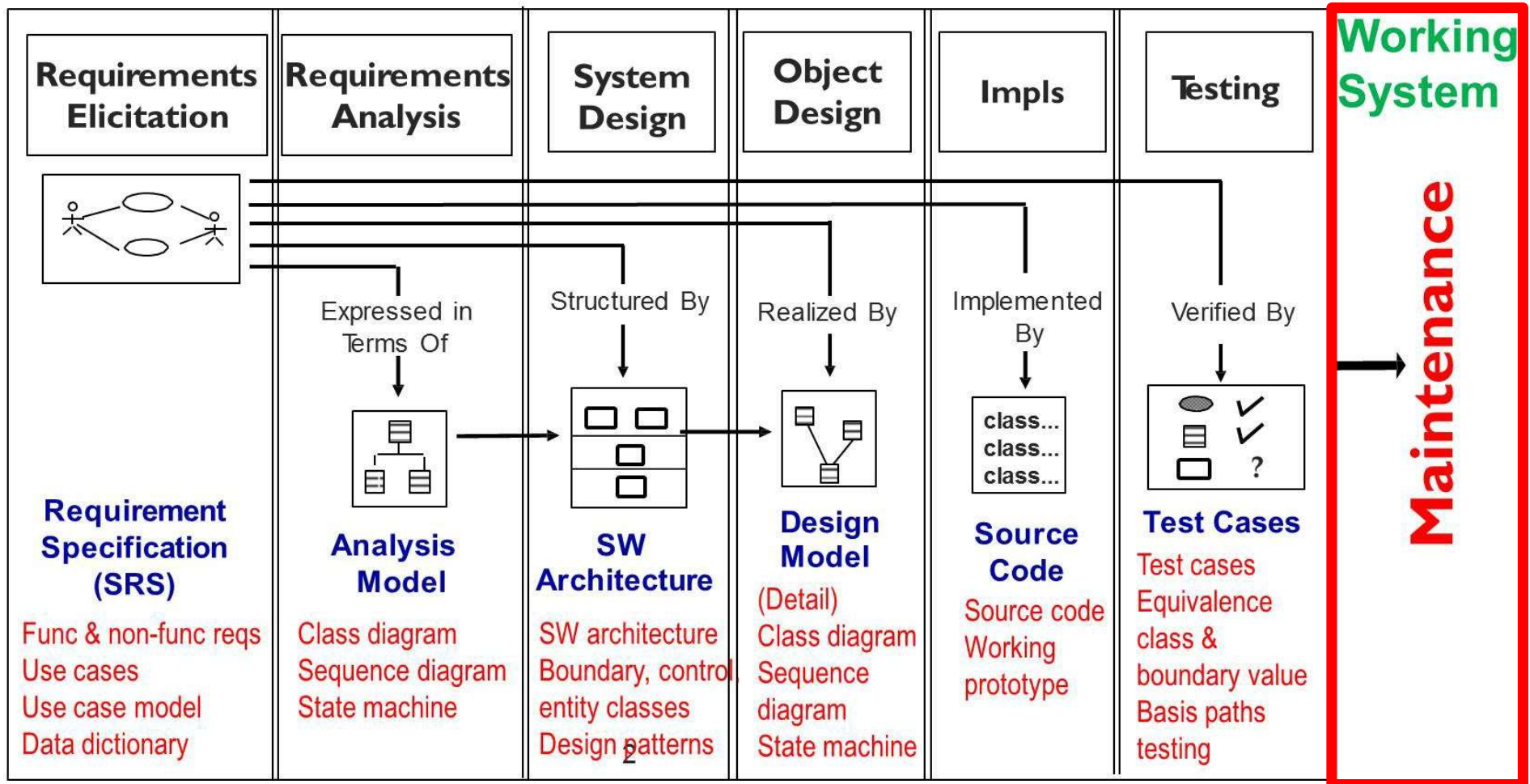
- ▶ Refactoring is a continuous process of improvement throughout the development and evolution process.
- ▶ Refactoring is intended to avoid the structure and code degradation that increases the costs and difficulties of maintaining a system.

# Program Code Improvements

- ▶ Situations (“*bad smells*”\*) in which the code of a program can be improved:
  - ▶ Duplicate code
  - ▶ Long methods
  - ▶ Large class
  - ▶ Temporary field
  - ▶ Switch (case) statements
  - ▶ Lazy class
  - ▶ Data clumping (same group of data reoccur in several places)
  - ▶ Tight coupling of two classes

\* Fowler, M. (1999). *Refactoring: Improving the Design of Existing Code*. Addison Wesley.

# Software Maintenance



# Quiz

## ▶ Quiz format:

- ▶ Date: 18 Nov 2021 (Thursday)
- ▶ Time: 12 noon
- ▶ Duration: 1 hour
- ▶ Answer **ALL** questions
- ▶ Five (5) MCQ
- ▶ Five (5) short-answer questions
  - ▶ Questions carry different marks
  - ▶ Questions may have sub-questions



# Quiz

- ▶ **Open Book**
  - ▶ **ONLY** paper materials (not limited to cheat sheet and textbook)
  - ▶ **NO** electronic devices allowed